Multilayer Perceptrons, Part 1

Advanced Machine Learning

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Learning objectives

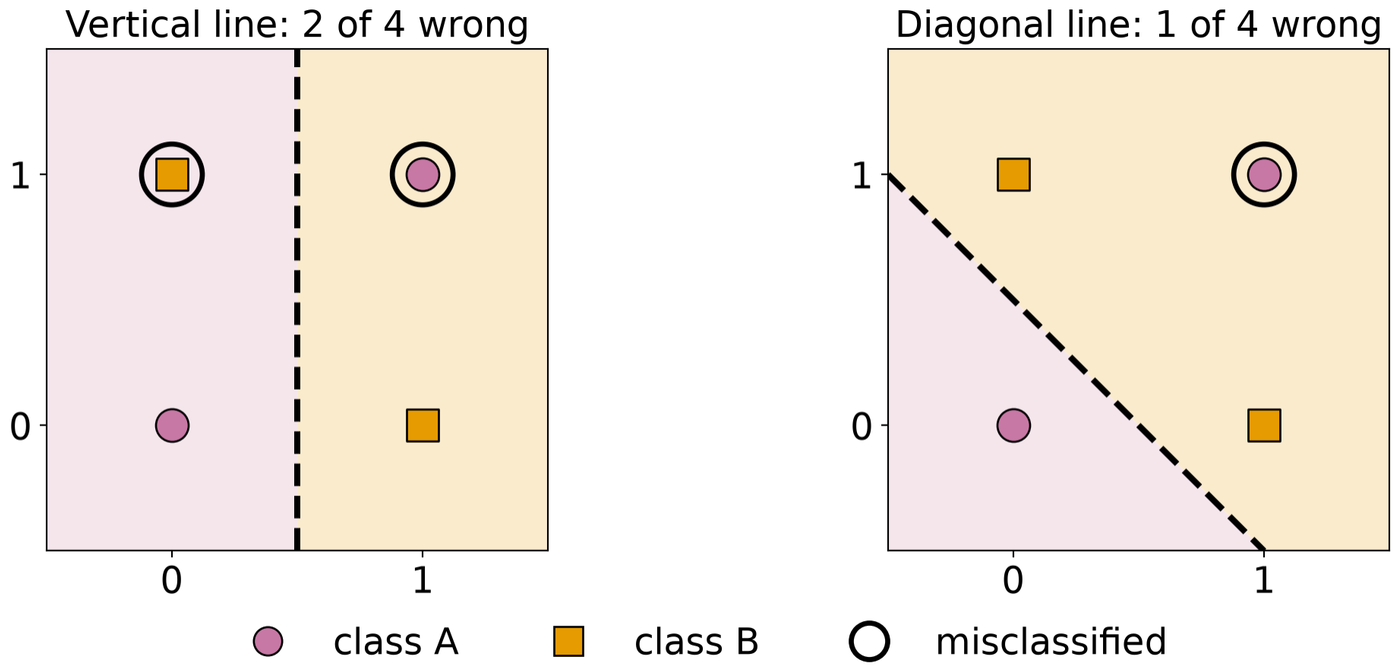
- Show why stacking linear layers alone doesn’t add any power
- Introduce hidden layers and activation functions as the fix
- Assemble the full MLP forward pass, and walk one example through it by hand

The limits of a linear model

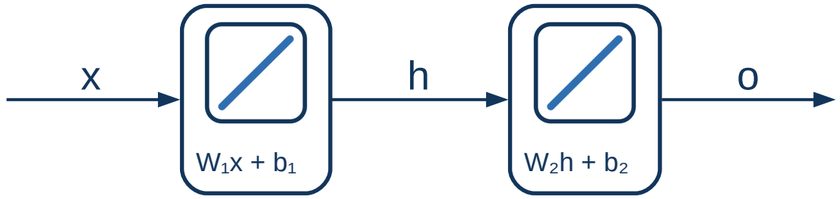
The Big Question

Does stacking more linear layers make a more powerful model?

A problem no linear boundary can solve



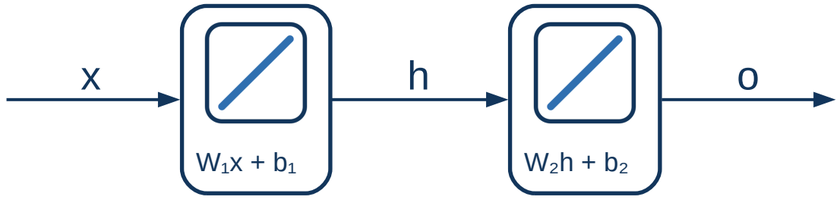
Try stacking two linear layers anyway



$$h = W_1 x + b_1, \qquad o = W_2 h + b_2$$

- The first layer's output h becomes the second layer's input
- Does the extra layer add power? Let's follow some real numbers through it

A smaller model, with numbers



$$W_1 = \begin{bmatrix} 1 & 1 \\ -1 & 2 \end{bmatrix}, \qquad W_2 = \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix}, \qquad x = \begin{bmatrix} 1.2 \\ 0.2 \end{bmatrix}$$

- Same two layers as above, now with actual values
- Biases set to 0 to keep the arithmetic short — they collapse the same way, as the general case will show

Follow one input through

$$h = W_1 x = \begin{bmatrix} 1 & 1 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} 1.2 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 1.4 \\ -0.8 \end{bmatrix}$$

$$o = W_2 h = \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1.4 \\ -0.8 \end{bmatrix} = \begin{bmatrix} 0.6 \\ -1.6 \end{bmatrix}$$

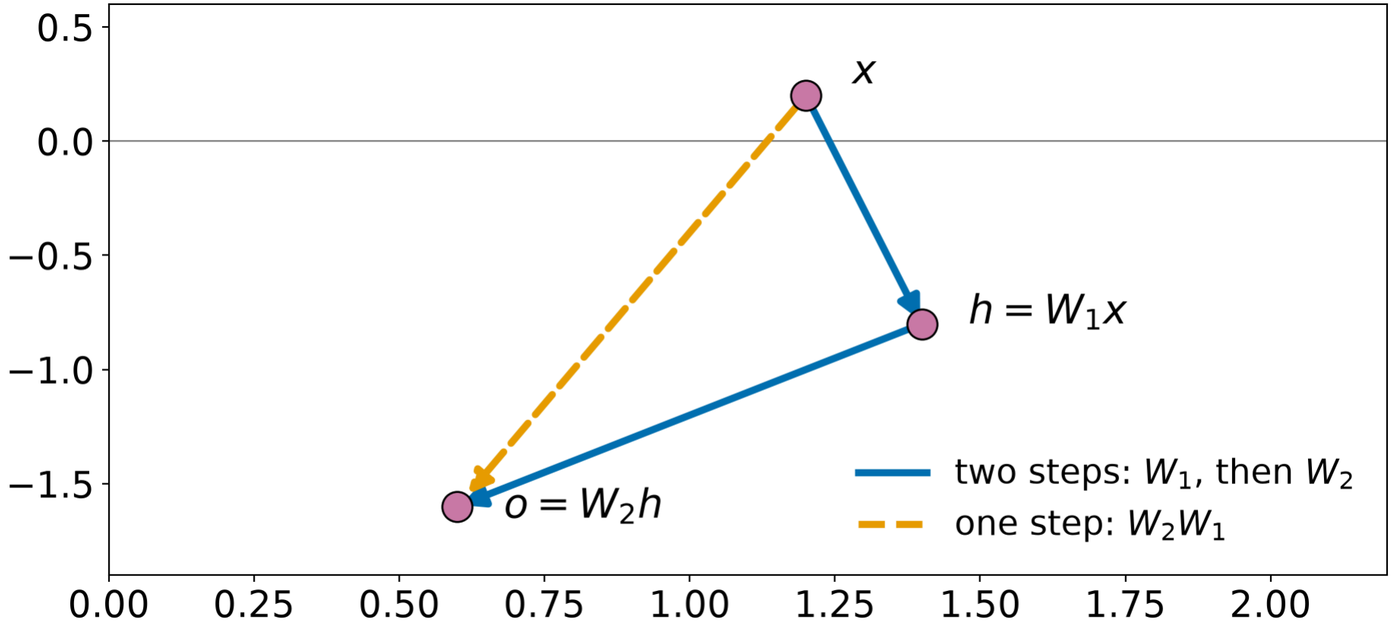
- Two steps: x becomes h, then h becomes o

Substitute one into the other

Since $h = W_1 x$, replace h in the second equation: $o = W_2(W_1 x) = (W_2 W_1)\, x$

$$W_2 W_1 = \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ -1 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 3 \\ -2 & 4 \end{bmatrix}, \qquad \begin{bmatrix} 0 & 3 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1.2 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 0.6 \\ -1.6 \end{bmatrix}$$

- One matrix, applied once, gives exactly the o that two steps did



What stacking gave us

In general, the same substitution gives:

$$o = W_2(W_1 x + b_1) + b_2 = \underbrace{(W_2 W_1)}_{W'} x + \underbrace{(W_2 b_1 + b_2)}_{b'} = W' x + b'$$

- Two stacked linear layers are exactly one linear layer with different parameters, however wide or deep
- So whatever a single linear layer can’t do — like the problem a few slides back — no stack of them can do either

Hidden layers and activation functions

The Big Problem

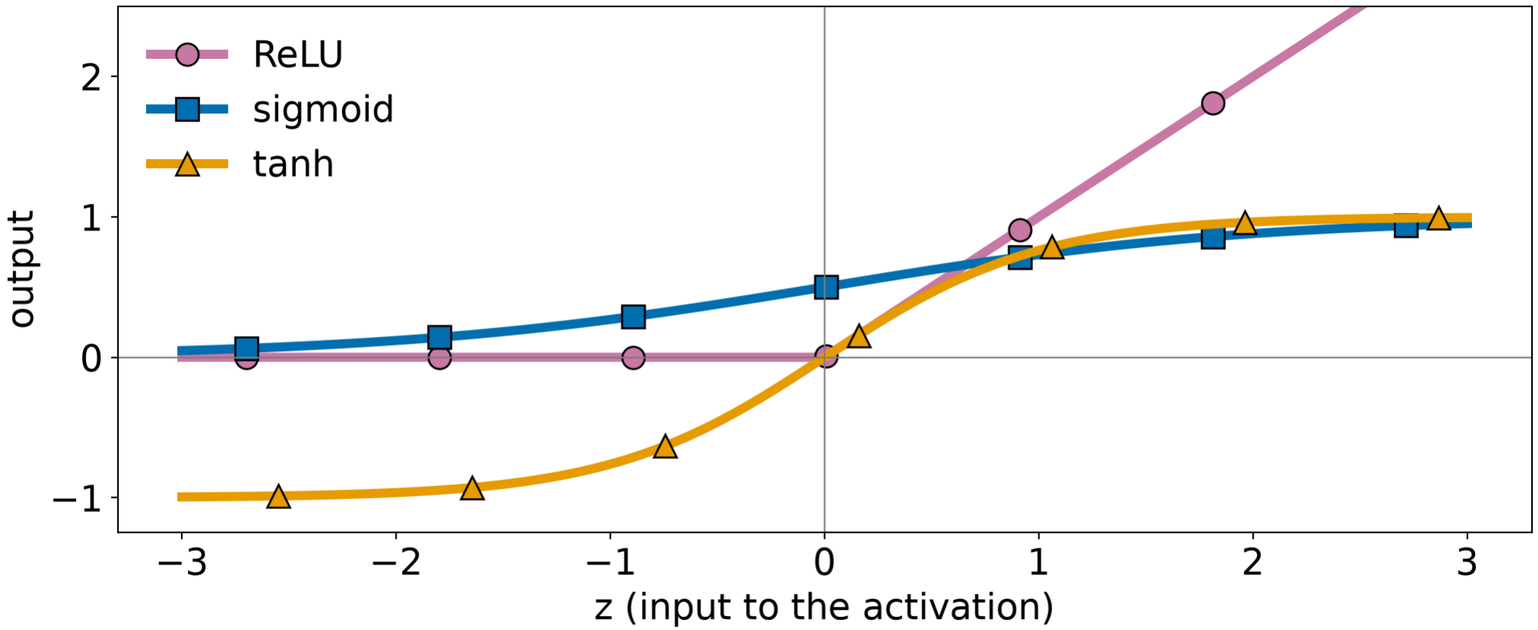
Linear-of-linear is still just linear.

We need more expressivity

The Big Idea

Non-linear functions make linear functions independent operations

ReLU, sigmoid & tanh



- Each one **bends** — its graph isn't a straight line, which is what a linear layer can never do
- That bend is what stops two layers from collapsing into one: the second layer sees a reshaped h, not a straight-line function of x

Breakdown: three activation functions

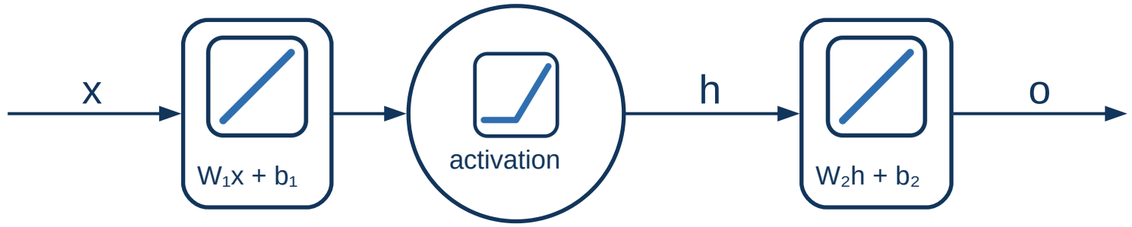
	output range	good	watch out
ReLU	$[0, \infty)$	cheap; grows for $z > 0$	zero for $z < 0$
sigmoid	$(0, 1)$	probability-like	saturates
tanh	$(-1, 1)$	centered at 0	saturates

Saturate: flatten out for large $|z|$, so the output barely changes as z changes — the slope is close to 0

Choosing one for now

- A flat curve means a tiny gradient, which means a very weak learning signal — more on why that matters in a later lecture
- tanh is just a stretched, shifted sigmoid ($\tanh z = 2\,\mathrm{sigmoid}(2z) - 1$): same S-shape and same saturation, but centered at 0
- This lecture uses **ReLU** — simplest to compute by hand, and its slope (next lecture) is either 0 or 1

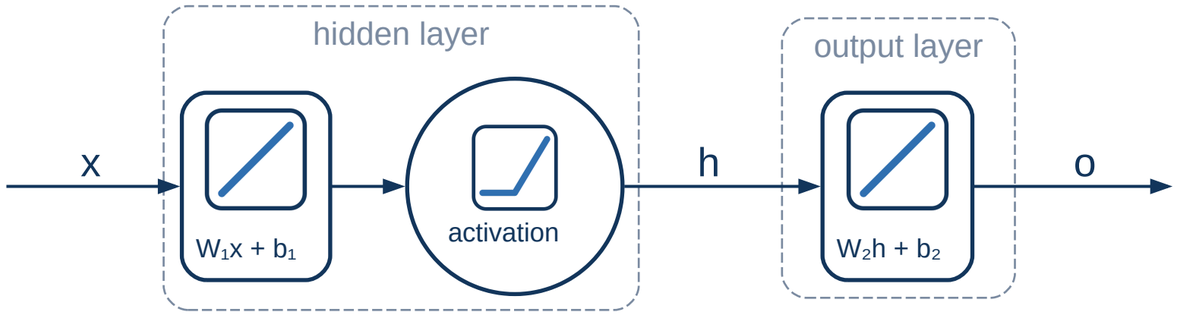
Putting an activation between the layers



$$h = \sigma(W_1 x + b_1), \qquad o = W_2 h + b_2$$

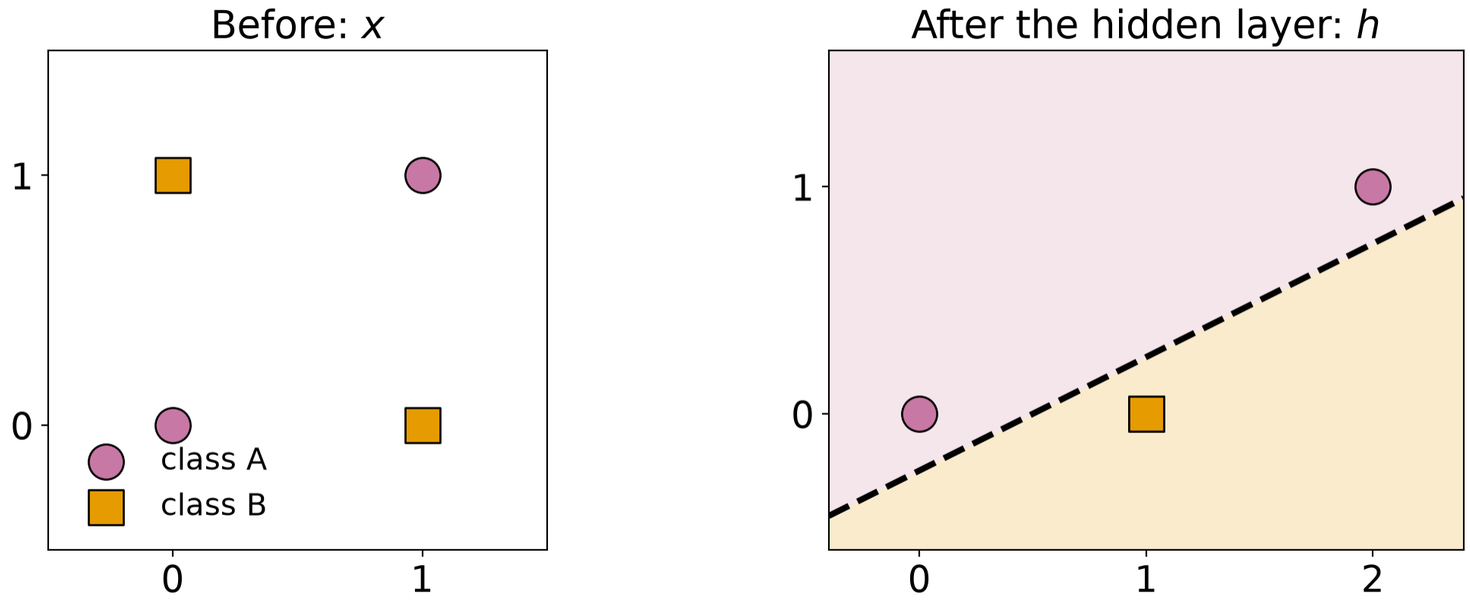
- σ stands for *any* activation function — ReLU, sigmoid, tanh — not only the sigmoid
- It's a function applied to values, not a layer with weights of its own — hence a circle, not a box
- Layer 2 no longer sees a straight-line function of x, but a reshaped version
- Call the value before σ acts $h_{\text{pre}} = W_1 x + b_1$, so that $h = \sigma(h_{\text{pre}})$

The hidden layer



- A **hidden layer** is a layer whose output feeds another layer instead of being the model's answer
- W_1 is the first layer of the model, yet hidden — the outside world sees only x going in and o coming out

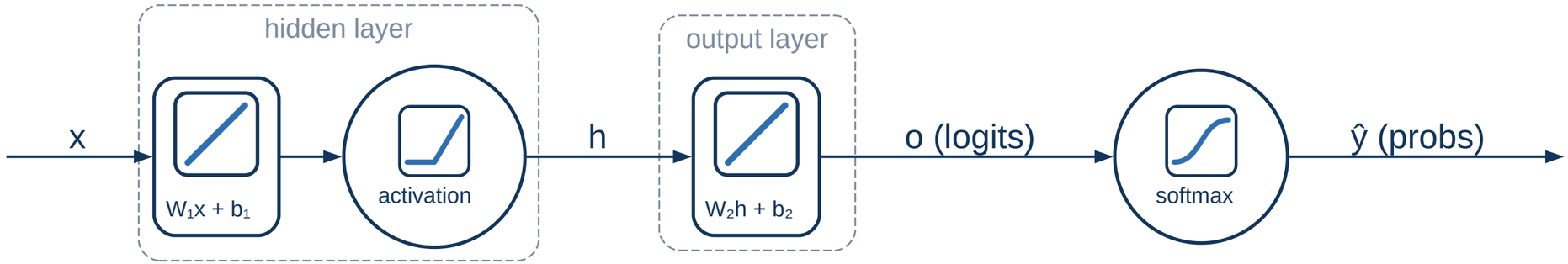
What the hidden layer buys us



- The hidden layer, $h_1 = \mathrm{ReLU}(x_1 + x_2)$ and $h_2 = \mathrm{ReLU}(x_1 + x_2 - 1)$, moves the points so that one straight line separates them
- The output layer draws that line; a linear model on x alone never could

The full forward pass

The full picture



$$h = \sigma(W_1 x + b_1), \qquad o = W_2 h + b_2, \qquad \hat{y} = \mathrm{softmax}(o)$$

- Hidden layer, output layer, softmax — applied one after another. This whole chain is what “forward pass” means for a network with a hidden layer

Worked example: our fruit, one layer deeper

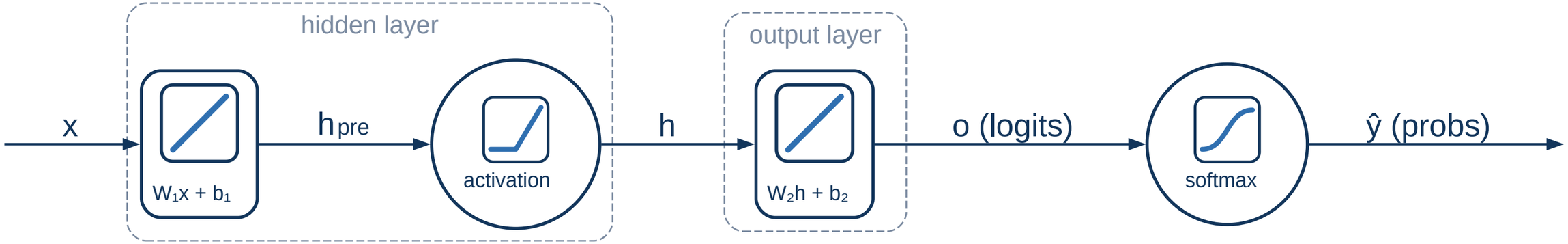
Adapting our example

Think back to last lecture: classifying a piece of fruit as apple, banana, or orange from its weight and color score. Now we put a hidden layer in front of that classifier.

$$x = \begin{bmatrix} 1.2 \\ 0.2 \end{bmatrix}, \qquad \text{true class} = \text{banana}$$

- Weight is scaled to hundreds of grams; color score runs from 0 (green) to 1 (red)
- The hidden layer has two values in it; the output layer still scores three classes

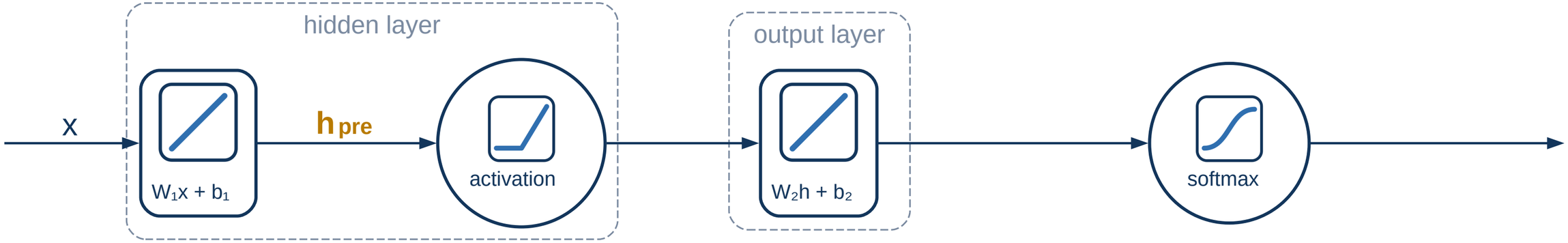
Our model, with its weights



$$W_1 = \begin{bmatrix} 1 & 1 \\ -1 & 2 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \qquad W_2 = \begin{bmatrix} 0.5 & 1 \\ -0.3 & 2 \\ 0.2 & -1 \end{bmatrix}, \quad b_2 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

- Rows of W_2 are apple, banana, orange; we're partway through training, so these are the current values

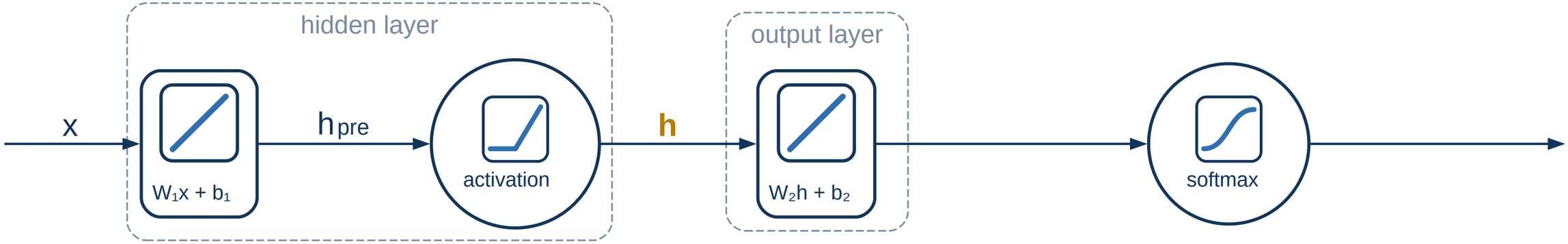
Step 1: the hidden layer's pre-activation



Call the value before the activation h_{pre}, so that $h = \sigma(h_{\text{pre}})$:

$$h_{\text{pre}} = W_1 x + b_1 = \begin{bmatrix} 1 & 1 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} 1.2 \\ 0.2 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1.4 \\ -0.8 \end{bmatrix}$$

Step 2: apply ReLU



ReLU acts on the whole vector at once, entry by entry:

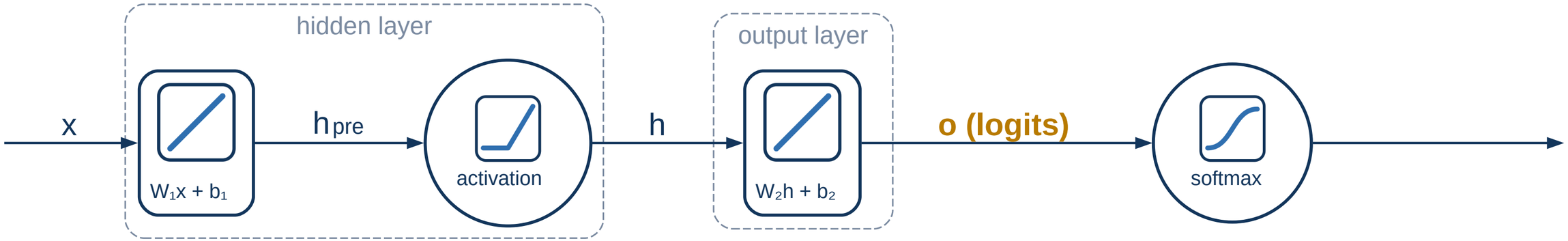
$$h = \mathrm{ReLU}\left(\begin{bmatrix} 1.4 \\ -0.8 \end{bmatrix}\right) = \begin{bmatrix} \max(0,\ 1.4) \\ \max(0,\ -0.8) \end{bmatrix} = \begin{bmatrix} 1.4 \\ 0 \end{bmatrix}$$

Important

Notice how this means h_2 has no contribution, because it was negative! Now the output will only use h_1.

What impact might that have on it's contribution to loss?

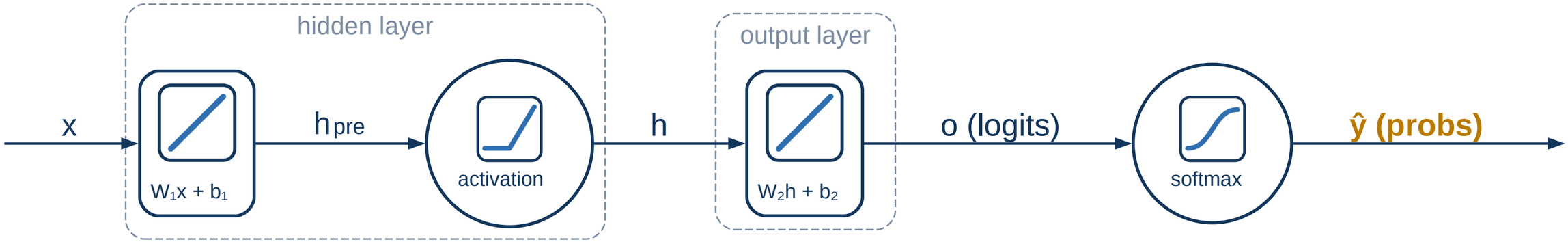
Step 3: the output layer



$$o = W_2 h + b_2 = \begin{bmatrix} 0.5 & 1 \\ -0.3 & 2 \\ 0.2 & -1 \end{bmatrix} \begin{bmatrix} 1.4 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.7 \\ -0.42 \\ 0.28 \end{bmatrix}$$

- Because $h_2 = 0$, the second column of W_2 contributed nothing this time

Step 4: softmax and the loss

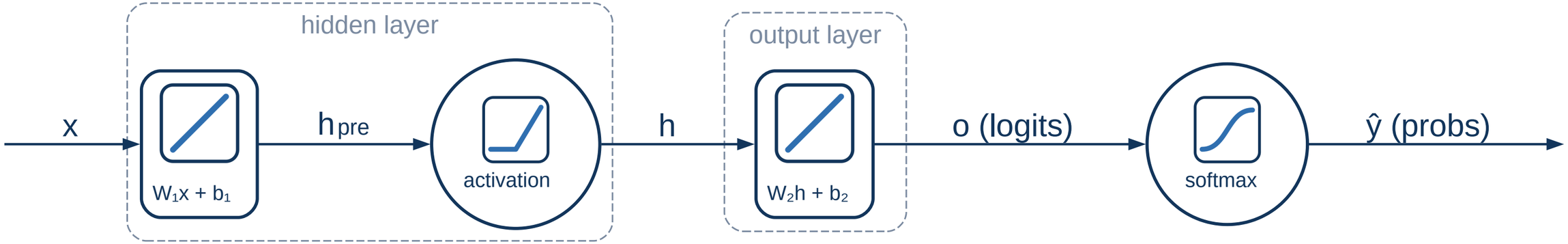


$$\hat{y} = \text{softmax}(o) = [0.5042,\ 0.1645,\ 0.3313]$$

$$L = -\log(\hat{y}_{\text{banana}}) = -\log(0.1645) \approx 1.8048$$

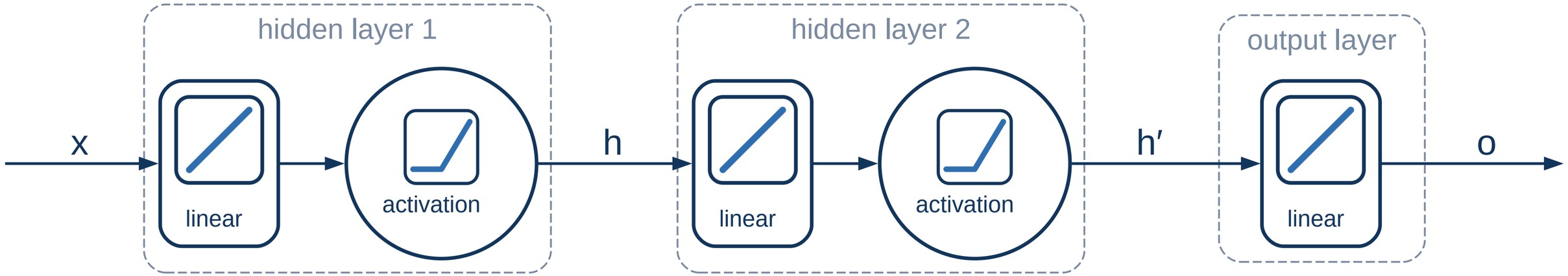
- Apple currently wins, even though the true label is banana — the model is still partway through training
- One layer deeper, same forward-pass mechanics: every step is still “apply this function, then the next”

The forward pass, at a glance



x	h_{pre}	h	o	$\hat{y}$
$[1.2,\ 0.2]$	$[1.4,\ -0.8]$	$[1.4,\ 0]$	$[0.7,\ -0.42,\ 0.28]$	$[0.50,\ 0.16,\ 0.33]$

As many hidden layers as we like



- **0 hidden layers**: our base case — the plain linear model from before
- **1 hidden layer**: this lecture's model
- **1000 hidden layers**: also allowed, though very deep models start running into problems of their own — more on those in a later lecture

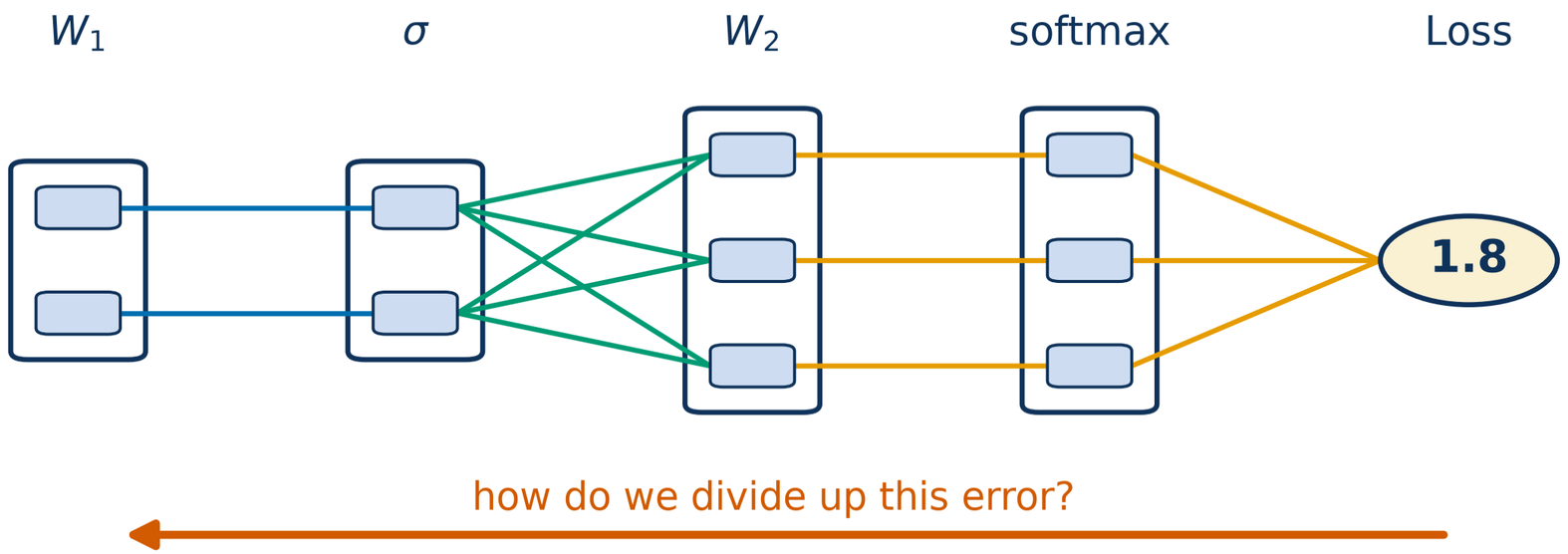
Same model, more in the middle

```
class Model(nn.Module):
    def forward(self, x):
        h = activation(self.layer1(x))
        # ... more model here -- we just need to pass values forward
        return self.layer2(h)

loss = loss_fn(model(x), y)   # forward pass
loss.backward()               # backward pass
```

- The idea of the model hasn't changed: a **forward pass** turns x into a prediction and a loss, and a **backward pass** turns that loss into gradients for training
- Hidden layers add depth, not a new kind of training — the backward pass through that extra step is what's coming next lecture

What comes next



- The loss, 1.8, shows up only at the very end — but every weight in W_1 and W_2 helped produce it
- Next lecture: how much of that error does each one deserve? Dividing it up is what the chain rule does for us

Summary

Study guide: multilayer perceptron

Input	a feature vector x
Output	logits o, then class probabilities $\hat{y}$ via softmax
Parameters	W and b for each layer: W_1, b_1, W_2, b_2
Loss	cross-entropy, unchanged
Special	hidden layers: $h = \sigma(W_1 x + b_1)$ between input and output
Why it matters	the nonlinearity learns what a linear model can't
Watch out	ReLU outputs of 0 pass nothing on; very deep networks train poorly